

Reinforcement Learning and Dynamic Optimization: **Final Project - Poker Playing Agent**

Pavlos Horn – 2022030203

Giorgos Vogiatzis – 2022030069

Περιεχόμενα

Introduction	1
LeDuc Hold'em	1
Texas Hold'em	6

Introduction

In this project we implemented several reinforcement learning agents that played two variations of poker: Leduc and Texas limit Hold'em. For both game versions, we initially trained agents against specific, fixed baseline strategies, then we developed agents that trained against the initial ones. The most “advanced” of our agents were sent to a “showdown” competition against the trained agents of other students. For Leduc poker, a model-free, tabular Q-learning algorithm was implemented. For Texas Hold'em, the same algorithm was rejected as the number of model representation states is too large. Instead, an approximate method with neural network assistance was used. Below, we will analyze the process and chosen parameters of the agents' training for both versions of the game.

Note: For Leduc poker, we implemented our own environment from scratch, and we also created a custom baseline opponent. For Texas Hold'em, we based our build on RLCard's environment.

LeDuc Hold'em

The rules of this variation are simple. Two players each hold a card {'J', 'Q', 'K'} and place their bets in two rounds: a pre-flop and a post-flop round. For the pre-flop round, no table cards are known, while for the post-flop round there is one public card on the table. Player resources are unlimited (i.e. there is no budget), but raising is fixed in terms of amount (+2 chips in pre-flop round, +4 in post-flop). The custom environment, as well as the training and testing process is provided in the notebook (“LeDuc_Poker.ipynb”). The learning agent implementation is provided in “Poker_Agent.py”, and the fixed

opponent baselines are included in “Player_Baselines.py”. The baselines are namely the following:

- ✓ Pal-Bot Opponent: Custom bot created based on the observed behavior of our dear **pal**, Mitsos **Palaiologos**. Bluffs smartly and plays aggressively.
- ✓ Random Opponent: Uniformly chooses among legal actions.
- ✓ Limping Opponent: Plays passively. Doesn’t fold but doesn’t raise either.
- ✓ Tight Opponent: Plays according to generally “good” poker strategies. Folds more often with weak hands, raises with stronger hands, and rarely bluffs.
- ✓ Loose/Aggressive Opponent: Never folds and raises whenever possible/legal.
- ✓ Honest-Strength Opponent: Aggressive depending on the quality of the cards.

Q-learning happens inside `Poker_Agent.learn()`. The agent learns based on storing $Q(s,a)$ values in a Q-table and choosing actions **a** that maximize the reward $R(s,a,s') = Q(s,a)$ when the game is in state **s**. Depending on the agent training, the Q-table has different q-values, meaning that agents trained against opponent style “a” do not learn to play “good” poker, but rather learn to beat opponent “a” in poker. Agents trained against “a” might not do so well against an opponent with strategy “b”, as demonstrated below:

	Pal_Bot	Random	Limping	Tight	Aggressive	Honest	Mixed
Q-Pal_Bot	0.58	0.84	0.10	0.27	0.08	0.15	0.15
Q-Random	-0.54	1.38	-0.82	-0.23	0.71	-0.32	-0.32
Q-Limping	0.32	0.99	0.22	0.38	0.16	0.37	0.35
Q-Tight	0.20	0.95	0.28	0.58	0.28	0.41	0.42
Q-Aggressive	-0.17	1.11	-0.39	-0.12	0.73	-0.07	-0.05
Q-Honest	0.10	0.95	0.17	0.21	0.21	0.42	0.43

Here, rows represent the q-learning agents that were tested against a specific baseline opponent, and columns represent the match-ups of these agents with all the baseline opponents. Values represent the mean reward of the Q-learning agent (measured in Big Blinds) over the course of the test. We can see that the agents are largely optimal when matched against the opponents they were trained for (except for Limping, but rewards with Tight are close so this was probably bad luck). However, there are many instances where agents trained against “a” lose to strategy “b” (signified by the negative rewards on the table).

The learning method goes as follows:

- ✓ The situation of the game is modelled and stored as a state in a state vector $\mathbf{s}$, containing information for:
 - The agent's private card (3 possible values)
 - The table's public card (4 possible values – 'J', 'Q', 'K', or None)
 - The action history (what has thus far happened in the game – 6 possible values)
 - The opponent strategy style (6 possible values)
 - Who plays first (Big-blind / Small – blind – 2 possible values)
- ✓ Possible actions are "Check", "Call", "Raise", "Fold" (4 possible values). So, the state-action pairs are $3 \times 4 \times 6 \times 6 \times 2 \times 4 = 144 \times 4 = 576$. The quality of each state-action pair is called the $Q(s,a)$ /q-value of the pair and is stored in a 144x4 table called the Q-table of the agent.
- ✓ During training, the agent plays a number of games against its opponent and receives a round reward at the end of each game.
- ✓ Initially, the Q-table is filled with random values. At every betting round, the agent chooses (legal) action $\mathbf{act}$, for which: $Q(\mathbf{s}, \mathbf{act}) = \max_a \{Q(\mathbf{s}, \mathbf{a})\}$.
- ✓ Depending on the round reward $\mathbf{r}$ received, the quality of the action taken is updated:

$$Q(\mathbf{s}, \mathbf{act}) = (1 - \alpha)Q(\mathbf{s}, \mathbf{act}) + \alpha \times Target,$$

where:

$$Target = r + \gamma \times \max_{\mathbf{act}'} \{Q(\mathbf{s}, \mathbf{act}')\}$$

- ✓ Parameters α , γ dictate the training quality. Specifically:
 - γ is the same foresight parameter as the one used in MDP Bellman equations. We want γ to be close to 1 so we can maximize long-term rewards, which means that we set $\gamma = 0.99$.
 - α is a sensitivity parameter for tampering with q-values. The more we choose an action, the smaller α becomes, according to:

$$\alpha = \max \left\{ 0.01, \frac{20}{20 + N(\mathbf{s}, \mathbf{act})} \right\},$$

where $N(\mathbf{s}, \mathbf{act})$ is a counter of how many times action $\mathbf{act}$ has been taken. This means that each action has its own α -value, and the more it is updated the closer it gets to "locking" to its optimal q-value. Locking is not absolute, but $Q(\mathbf{s}, \mathbf{act})$ gets more "resilient" over time, in the sense that the agent has pretty much made up its mind for the quality of this particular state-action pair (unless something completely groundbreaking happens and Target is severely affected).

Below, the training process versus baseline opponents is explained:

- ✓ The agent is set to play a total of 500000 games against the baseline opponent to ensure that there is enough time for the algorithm to converge. An attempt at a dynamically formulated training with a “while not converged” condition was made, but training took too long and we had to revert back to a fixed number of episodes per agent.
- ✓ Each training game provides the agent with the round reward, and the agent updates its q-table based on it.
- ✓ The q-table of each agent gradually converges by decaying the α and ϵ parameters: α decays as explained above, and ϵ decays exponentially according to:

$$\epsilon(t) = \max \{0.01, 0.99999^t\}$$

ϵ exists to ensure some level of exploration at all stages of the training process. At the latter stages of its training, the agent will lock itself in specific state-pair actions but will leave a 1% margin of random exploration. Inversely, the initial q-values are random and do not give useful information about the optimality of each action. So, exploration is warranted. We do not want ϵ to decrease too quickly (0.99999^t), as this would prematurely “lock” the agent in suboptimal q-values. Additionally, we do not want ϵ to decrease too slowly (0.9999^t), as this would result in training heavily influenced by random exploration, which is also suboptimal.

The training process for the competitive agent is similar in structure, with the difference that it trains against several types of opponents. Our submitted agent was “Horn-y_Bot” and the thought behind the training process was the following:

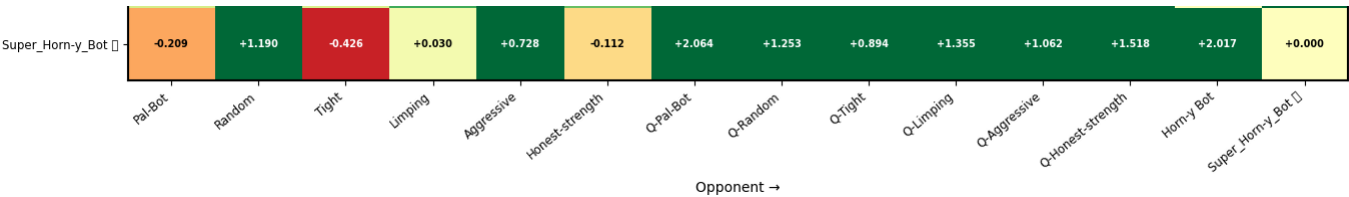
- ✓ Begin by training against “easy” fixed opponents to get an initial “direction” for the q-table (0-15% of training). The “easy” players are namely ‘Aggressive’ and ‘Random’. They were the opponents against which the highest rewards were yielded.
- ✓ Continue by training against “stronger” fixed opponents to start improving the semi-tuned q-table (15-60% training). “Strong” fixed opponents are the ones remaining in the list, namely “Pal-Bot”, “Limping”, “Honest-strength” and “Tight”.
- ✓ Conclude the process (60-100% of training) by training against a mix of fixed opponents (train against such an opponent at every round with medium probability = 35%), other pre-trained Q-learning specialists (high probability = 40%), as well as frozen snapshots of Horn-y_Bot’s past self (low probability = 25%).

The goal was to create an agent that would do well against any opponent that had been created at that point, and while this did work, Horn-y_Bot didn’t do too well in the

showdown arena. The reason was probably that there needed to be an extra agent with training against a fully trained Horn-y_Bot. This bot would be a specialist in defeating other competitive agents instead of striving to be a universal, conservative “Jack of all trades”. We attempted to make such an agent after the showdown competition, and we ended up with the “Super_Horn-y_Bot”. The Super_Horn-y_Bot may not yield the best results against every single opponent but IS the greatest when it comes to fighting against previous Q-learning agents, including “Horn-y_Bot”. In its final phase (which is now the longest – 35-100% of training), its training consists of:

- ✓ 50 % Horn-y Bot
- ✓ 20 % specialists
- ✓ 15 % fixed styles
- ✓ 15 % self-play

A custom showdown tournament was held after the competition against the other students was concluded. The goal was to evaluate the performance of Super_Horn-y Bot. Below, the tournament leaderboard and a heat map for the individual performance of SHB is provided. We see that Horn-y Bot is the best all-around player, while Super_Horn-y specializes in defeating q-learners. There **were** some instances where SHB finished 1st in the rankings, so performance varies.



GRAND TOURNAMENT LEADERBOARD		
Rank	Agent	Avg BB/hand
1	Horn-y Bot	+0.9966
2	Super_Horn-y_Bot ★	+0.8740
3	Q-Tight	+0.8520
4	Q-Limping	+0.7768
5	Pal-Bot	+0.7730
6	Q-Random	+0.7649
7	Q-Honest-strength	+0.7478
8	Q-Aggressive	+0.6502
9	Tight	+0.6340
10	Limping	+0.5990
11	Q-Pal-Bot	+0.5984
12	Aggressive	+0.4418
13	Honest-strength	+0.2887
14	Random	-0.2461

Texas Hold'em

For Texas Hold'em, tabular Q-learning was not feasible due to the vastly larger state space. The state is represented as a 72-dimensional float vector provided by RLCard's environment: the first 52 elements are a one-hot encoding of the cards (private and community), and the remaining 20 elements encode the betting history across four rounds. Instead of a Q-table, a Deep Q-Network (DQN) with a target network and experience replay was used to **approximate** Q-values over this continuous state space.

For environment compatibility, a custom wrapper (class LimitHoldemEnv in Environment.py) was built around RLCard's limit-holdem environment to provide a single-agent interface:

- ✓ The DQN agent always receives observations from its own perspective.
- ✓ The opponent's turns are handled automatically inside the wrapper.
- ✓ The four legal actions are: Call (0), Raise (1), Fold (2), and Check (3).
- ✓ The reward is 0 at every intermediate step and equals the hand payoff in big blinds at the end of each hand.
- ✓ The DQN agent is assigned to player 0; RLCard rotates blind positions between rounds, so the agent sees both small blind and big blind situations over the course of training.

Network Architecture and Algorithm

The Q-network (class QNetwork in Agent.py) is a three-layer MLP: $72 \rightarrow 256 \rightarrow 256 \rightarrow 4$, with ReLU activations and no output activation (raw Q-values). Standard DQN improvements were applied:

- ✓ Target network: a frozen copy of the online network, synced every 1,000 steps, stabilizing the Bellman targets.
- ✓ Experience replay: a circular buffer of 100,000 transitions (params: s, a, r, s', done, next_legal_mask), sampled in minibatches of 128.
- ✓ Illegal action masking: Q-values for illegal actions are set to $-\infty$ before both action selection and target computation, preventing the agent from wasting training time unlearning illegal moves.
- ✓ Vanilla DQN target: the target network evaluates next-state Q-values directly; illegal next actions are masked to $-\infty$ before taking the maximum q-value.
- ✓ The loss function is MSE, and gradient norms are clipped to 10. The optimizer is Adam with learning rate 1×10^{-4} (note: we tried a "better" loss function in

nn.SmoothL1Loss()) and tried lowering the clipping threshold to 1. However, after several other changes to the code, results were worsened, and we had to revert back to an older version).

Baseline Opponents

Four fixed-strategy baseline opponents were implemented, mirroring the Leduc setup. Namely:

- ✓ Random Opponent
- ✓ Limping Opponent
- ✓ Tight Opponent
- ✓ Aggressive Opponent

Training Pipeline

Training followed a three-stage pipeline, analogous to the Leduc Super_Horn-y_Bot approach:

- ✓ Stage 1 - Specialist agents: Four DQN agents were trained independently, each against one of the four fixed baselines for 100,000 episodes. These specialists learn strong but narrow strategies.
- ✓ Stage 2 - Curriculum agent (Horn-y Bot): A new DQN agent was trained against a random mix of the four specialists each episode, forcing it to generalize across all styles. Pre-built environments and pre-loaded specialists were reused each episode to avoid disk I/O overhead.
- ✓ Stage 3 - Final Boss (Horn-y DQN): A final agent was trained exclusively against the frozen curriculum agent, aiming to learn strategies that specifically exploit the curriculum agent's generalized policy.

Hyperparameters

- ✓ Learning rate $\alpha = 1 \times 10^{-4}$, discount $\gamma = 0.99$ (for far-sightedness)
- ✓ ϵ -greedy exploration: ϵ starts at 1.0, decays exponentially per episode, floored at 0.02:

$$\epsilon(t) = \max\{0.99995^t, 0.02\}$$

```
# Declining exploration for ~80% of the training horizon (H = 100k episodes).
#   - If decay was 0.9999,  $\epsilon$  would hit the floor limit too soon (~40% of the horizon).
#   - If decay was 0.99999,  $\epsilon$  would not hit the floor limit fast enough (within the horizon limit)
#   - With decay = 0.99995, we ensure smooth enough decaying while leaving enough training time for maximum exploitation
```

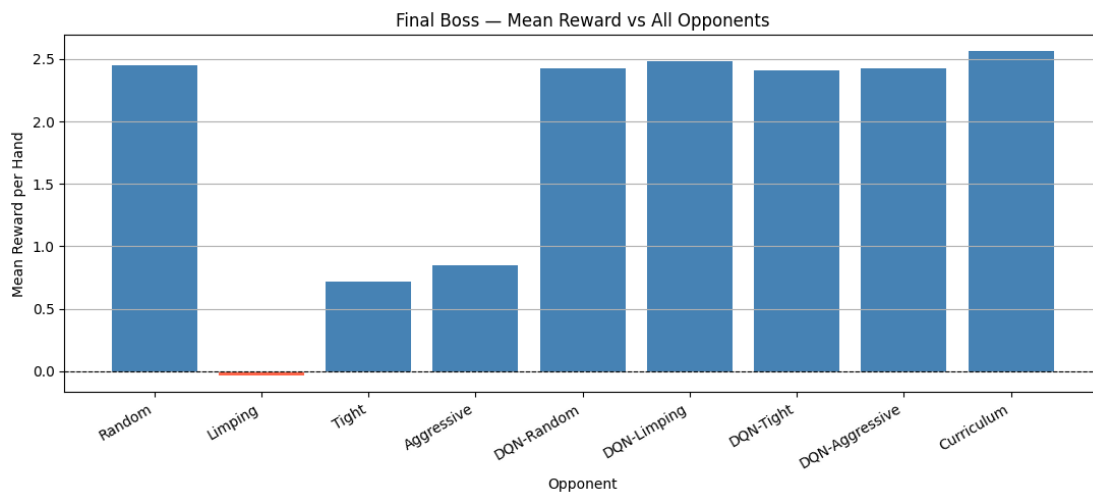
- ✓ Replay buffer capacity: 100,000 transitions; minimum buffer before learning: 1,000
- ✓ Target network sync: every 1,000 gradient steps; hidden layer width: 256

Results

Final mean round rewards per agent and opponent:

	Random	Limping	Tight	Aggressive	Mixed
<i>Random DQN</i>	2.79	0.68	0.05	1.13	1.20
<i>Limping DQN</i>	1.02	0.74	-0.57	0.14	0.32
<i>Tight DQN</i>	1.80	0.72	0.13	0.57	0.86
<i>Aggressive DQN</i>	2.12	0.75	-0.11	0.85	0.96

Final Boss / Horn-y_Bot rewards against every opponent:



Final Boss Mean Rewards per Opponent	
<i>Random</i>	+2.4478
<i>Limping</i>	-0.0361
<i>Tight</i>	+0.7153
<i>Aggressive</i>	+0.8516
<i>DQN-Random</i>	+2.4285
<i>DQN-Limping</i>	+2.4827
<i>DQN-Tight</i>	+2.4108
<i>DQN-Aggressive</i>	+2.4282
<i>Curriculum</i>	+2.5621

The submitted agent (“Horny_bot”) is the curriculum-trained DQN wrapped in a portable RLCARD-compatible interface (RLCARD DQNAgent). The wrapper translates RLCARD’s raw state dictionary into the 72-float vector and legal action mask expected by the DQN and implements set_player_id() so the agent correctly handles both seat positions during evaluation. The agent runs fully greedy ($\epsilon = 0$) at test time.